

AIML Assignment – Search Algorithms on Robot Path Finding

Name: Harshini Ramdas Bhandary

Division: A

Batch: A2

PRN: 202401040151

Roll No: 35

Assignment: AIML – Search Algorithms

Title: Performance evaluation of problem-solving search algorithms – Uninformed, Informed, Local Search and Constraint Satisfaction.

Aim: To implement and compare several Artificial Intelligence search techniques on a robot path-finding problem in a 2D grid, covering Uninformed Search, Informed Search, Local Search and Constraint Satisfaction / Optimization methods.

Objectives

1. To implement the uninformed strategies BFS, DFS, Depth-Limited Search (DLS) and Iterative Deepening DFS (IDDFS) that move the robot from the start state to the goal state.
2. To implement A* Search as an informed strategy guided by a heuristic function.
3. To implement the local search methods Hill Climbing and the Genetic Algorithm for building a path.
4. To model path finding as a Constraint Satisfaction Problem and solve it with backtracking search.
5. To compare all algorithms on path length, number of nodes explored and execution time.
6. To identify the strengths and weaknesses of each search strategy on the same problem.
7. To study how the choice of algorithm changes the efficiency and quality of the robot's path.

Theory: Search Algorithms in Artificial Intelligence

In Artificial Intelligence, a search algorithm finds a sequence of actions that takes an agent from an initial state to a goal state. The problem is modelled as a state space: each state is one possible configuration, and each transition is an action that moves the agent between states.

Search methods fall into four broad groups: Uninformed Search, Informed Search, Local Search and Constraint Satisfaction.

1. Uninformed Search

Uninformed (blind) search has no knowledge of how far a state is from the goal. It explores using only the problem definition.

- **Breadth-First Search (BFS):** expands all states at one depth before moving to the next. It returns the shortest path when every step costs the same.
- **Depth-First Search (DFS):** follows one branch as deep as possible, then backtracks. It uses little memory but does not guarantee the shortest path.
- **Depth-Limited Search (DLS):** DFS with a fixed depth limit L . Nodes at depth L are not expanded, which stops infinite descent, but no solution deeper than L can be found.
- **Iterative Deepening DFS (IDDFS):** runs DLS repeatedly with limits 0, 1, 2, and so on until the goal is found. It combines the low memory of DFS with the shortest-path guarantee of BFS, at the cost of re-expanding shallow nodes.

2. Informed Search

Informed (heuristic) search uses extra knowledge, a heuristic $h(n)$, to steer the search toward the goal.

- **A* Search:** ranks states by the actual cost so far plus the estimated remaining cost. With an admissible heuristic such as Manhattan distance, it always finds the optimal path:

$$f(n) = g(n) + h(n)$$

Here $g(n)$ is the cost from the start to state n , and $h(n)$ is the estimated cost from n to the goal.

3. Local Search

Local search keeps one or more candidate solutions and tries to improve them step by step, instead of exploring the whole state space systematically.

- **Hill Climbing:** begins at a start state and keeps moving to the neighbour with the best evaluation. It is simple and memory-light, but can get stuck at local maxima, plateaus, ridges or dead ends.
- **Genetic Algorithm (GA):** a population-based optimization method modelled on natural evolution. Candidate solutions are encoded as chromosomes and improved through population initialization, fitness evaluation, selection, crossover, mutation and formation of the next generation. The cycle repeats until an acceptable solution appears or the generation limit is reached.

4. Constraint Satisfaction

A Constraint Satisfaction Problem (CSP) has a set of variables, a domain of values for each, and constraints that decide which combinations are allowed.

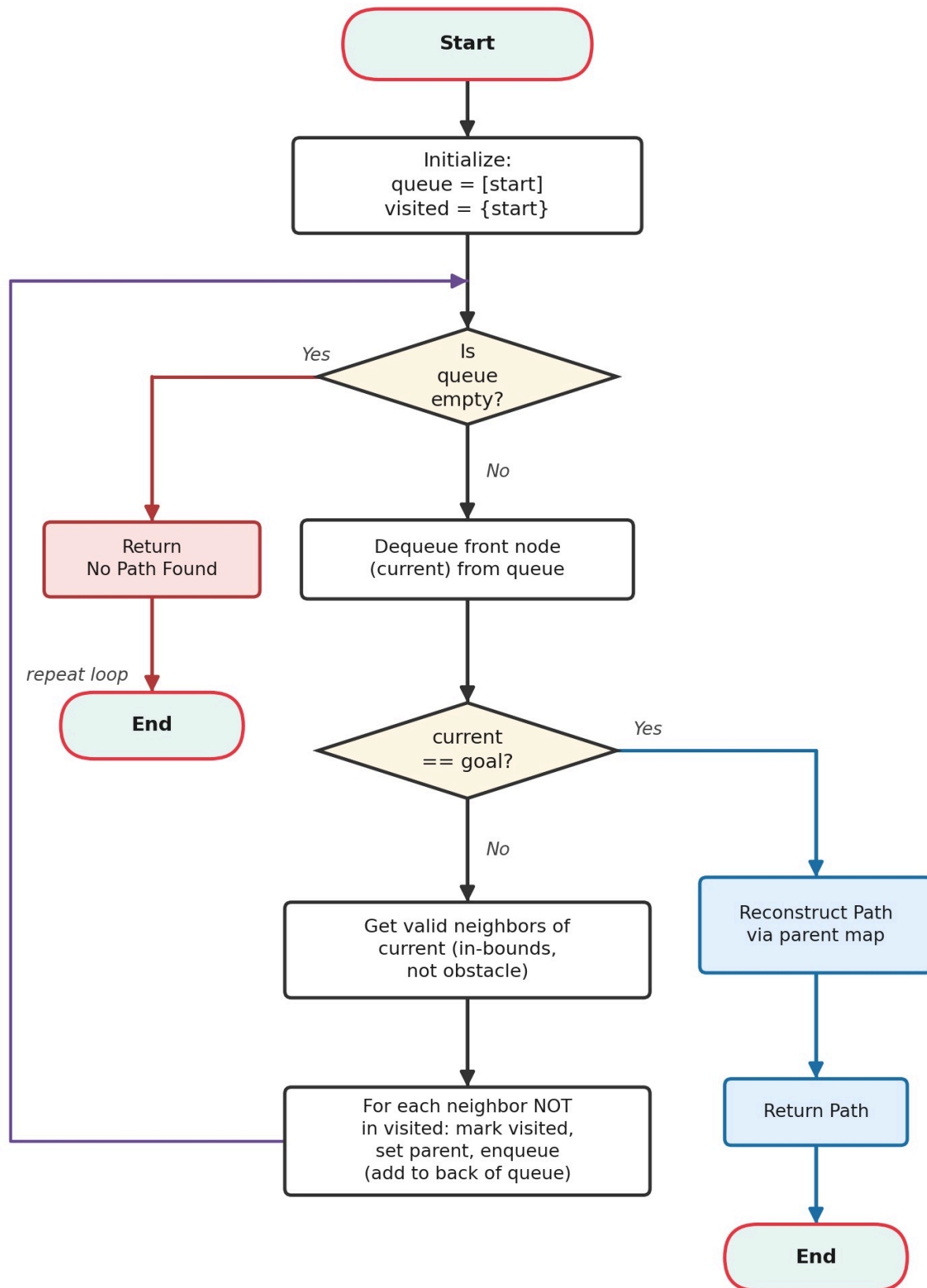
Robot path finding can be written as a CSP:

- **Variables:** $X_0, X_1, \dots, X_k$, where X_i is the cell the robot occupies at step i .
- **Domain:** every free cell in the grid.
- **Constraints:** X_0 = start and X_k = goal; each $X_{(i+1)}$ is adjacent to X_i ; no cell is used twice; the goal must still be reachable in the steps left.

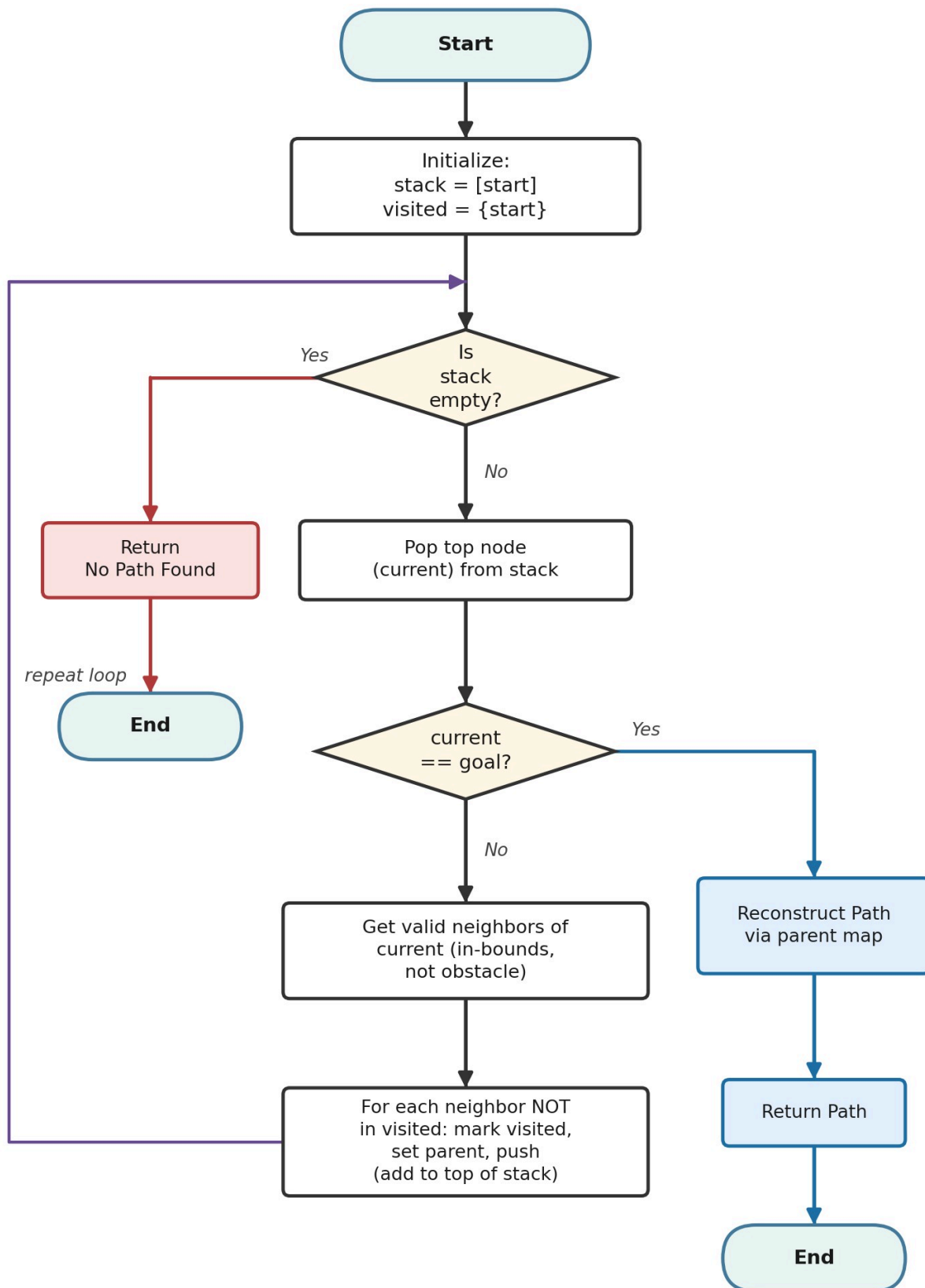
Backtracking search assigns the variables one at a time. When a value breaks a constraint it tries the next value, and when no value fits it undoes the previous assignment (backtracks). Checking the distance-to-goal constraint early (forward checking) prunes hopeless branches before they are explored.

Flowcharts

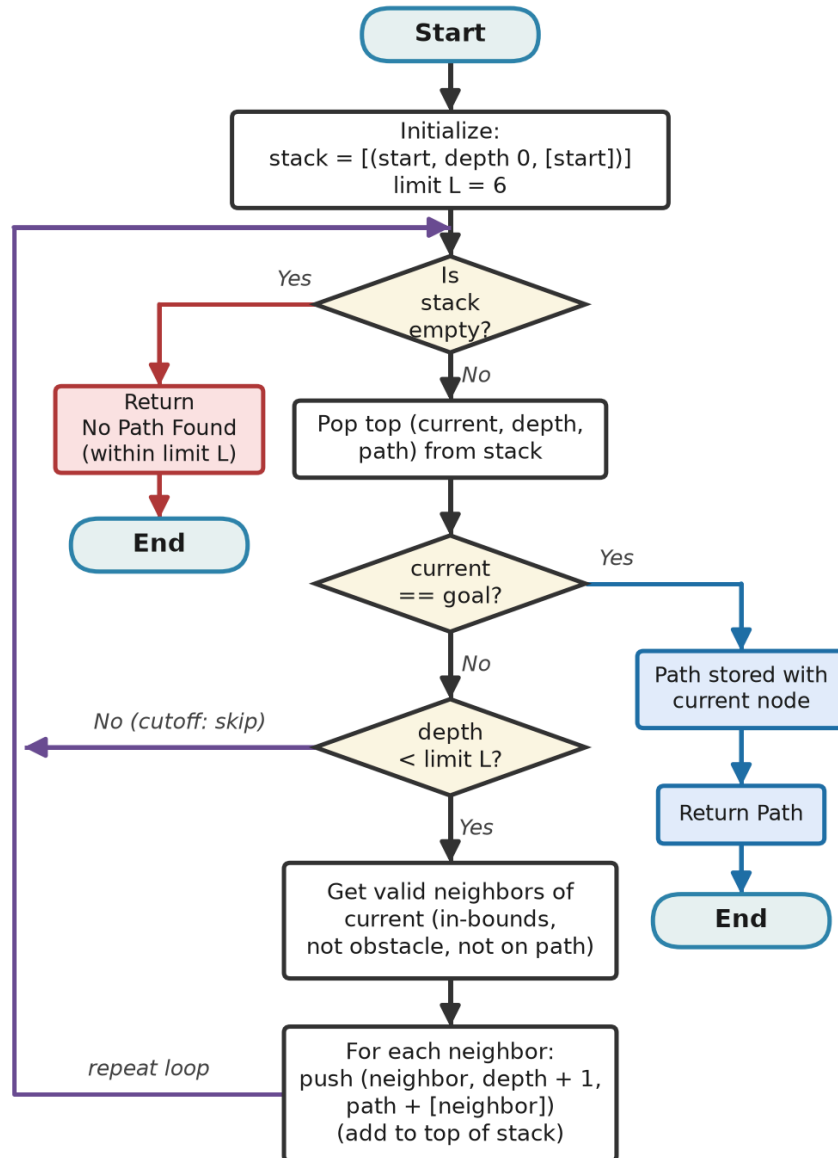
BFS (Breadth-First Search) – Uninformed



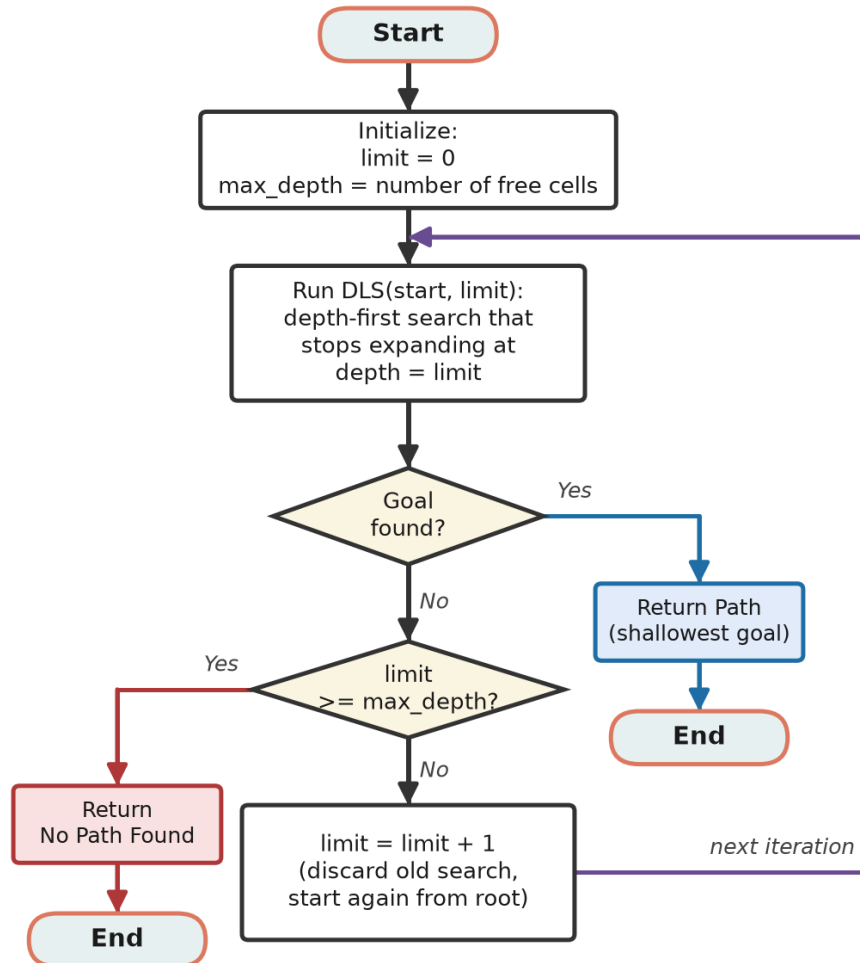
DFS (Depth-First Search) – Uninformed



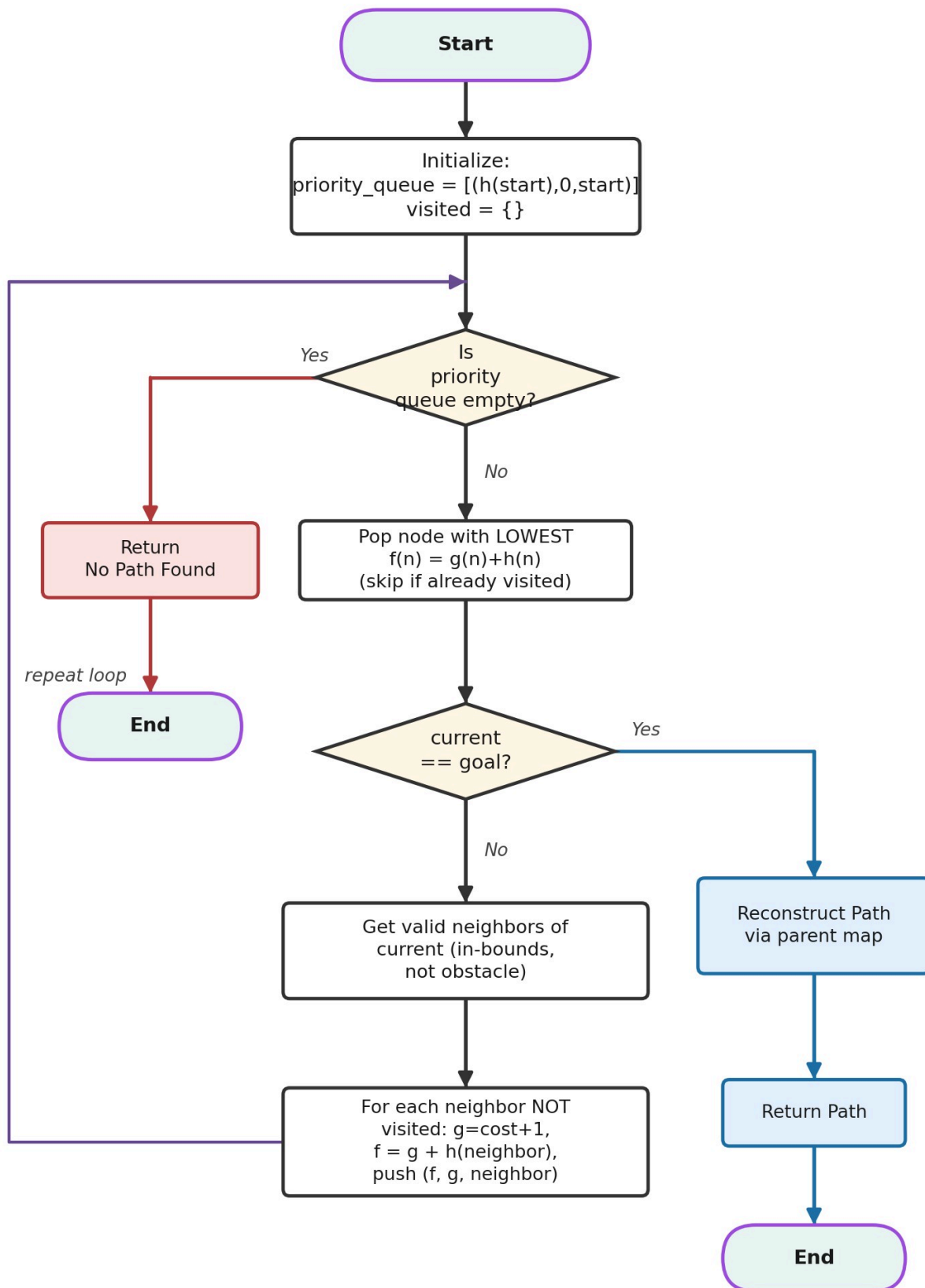
DLS (Depth-Limited Search) — Uninformed



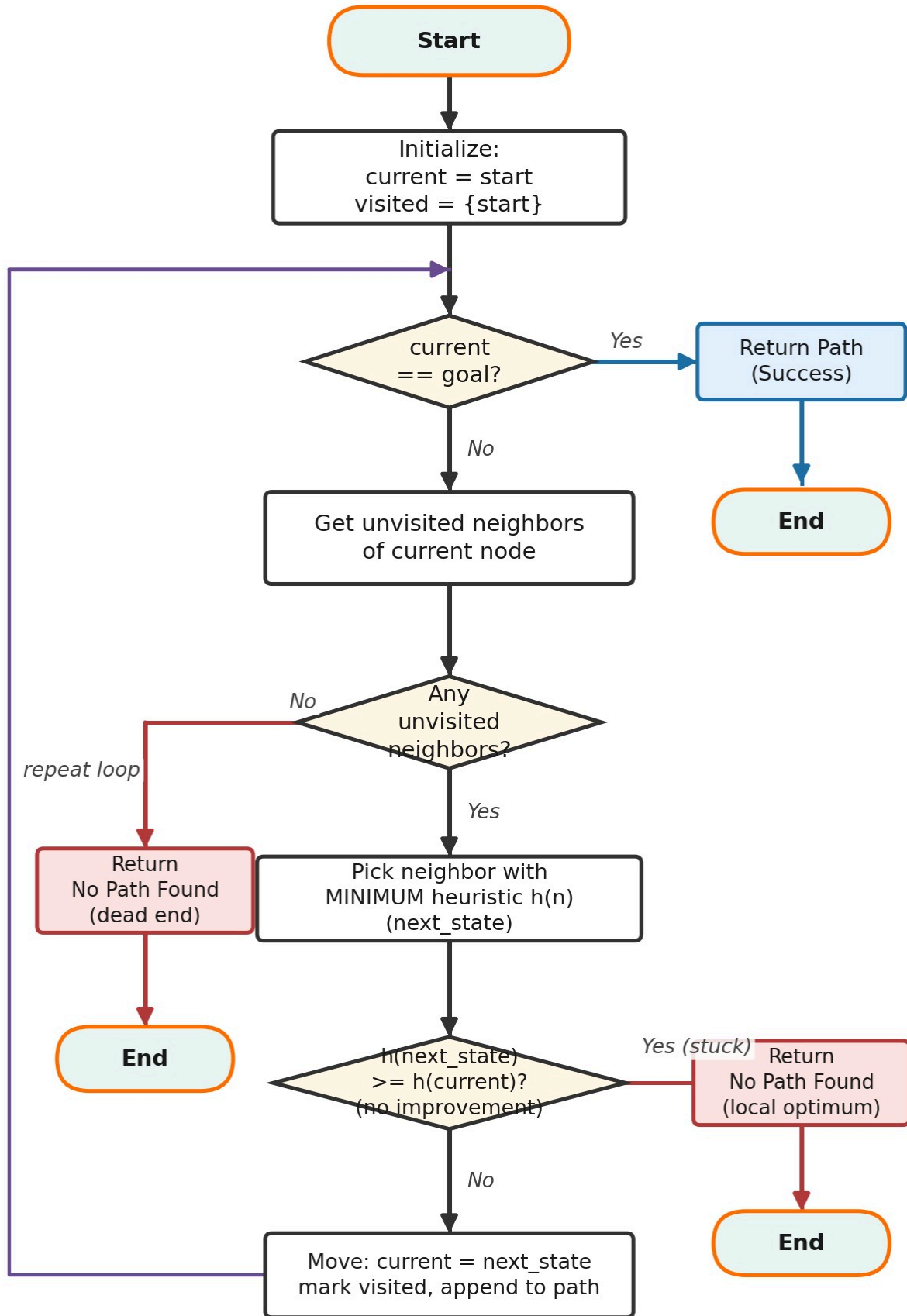
IDDFS (Iterative Deepening DFS) – Uninformed



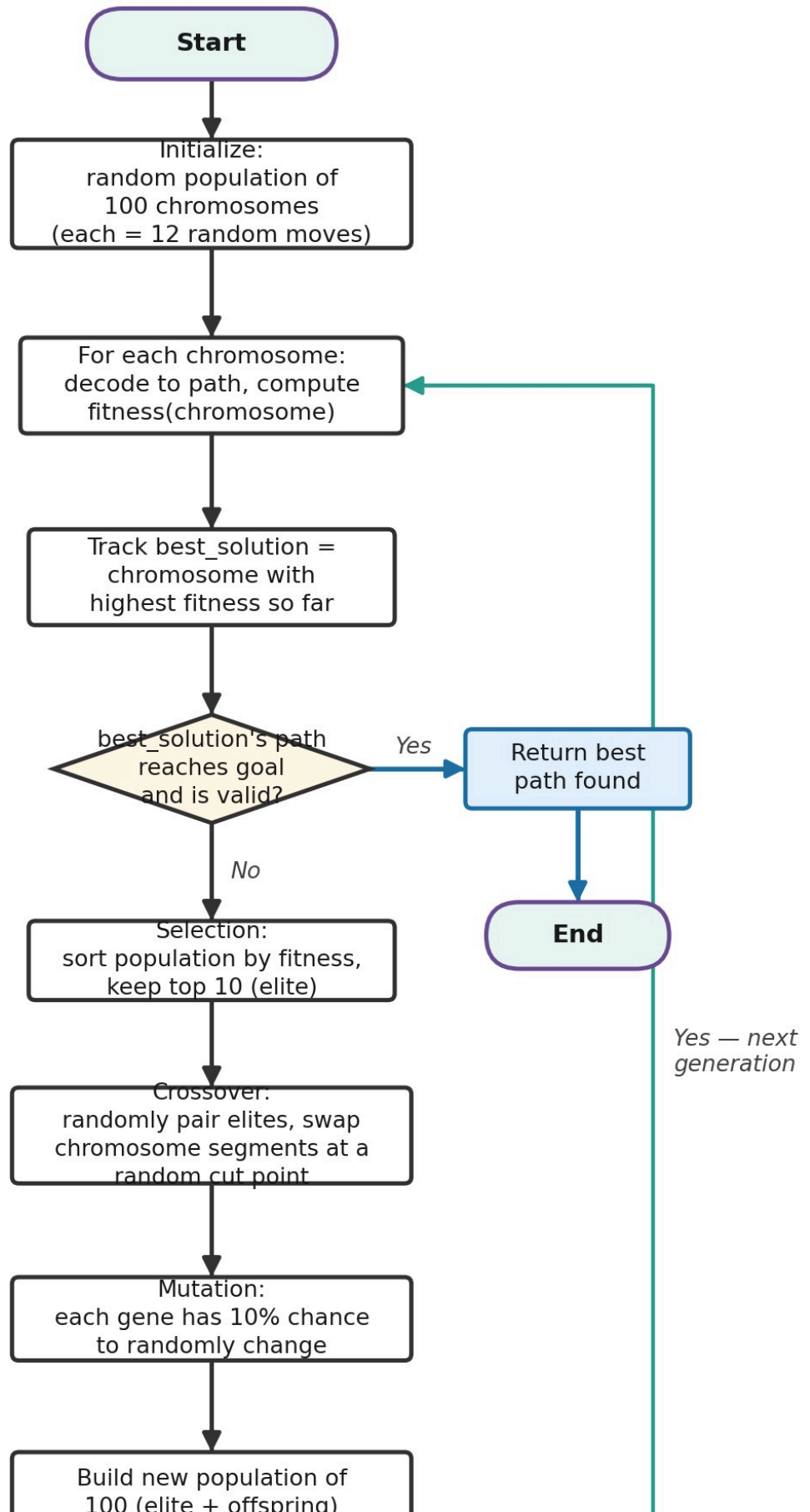
A* Search — Informed

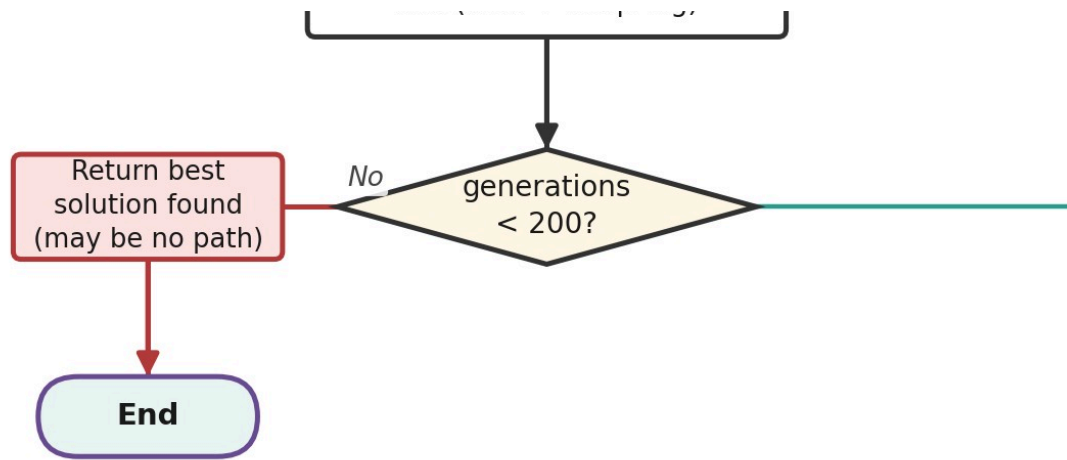


Hill Climbing – Local Search

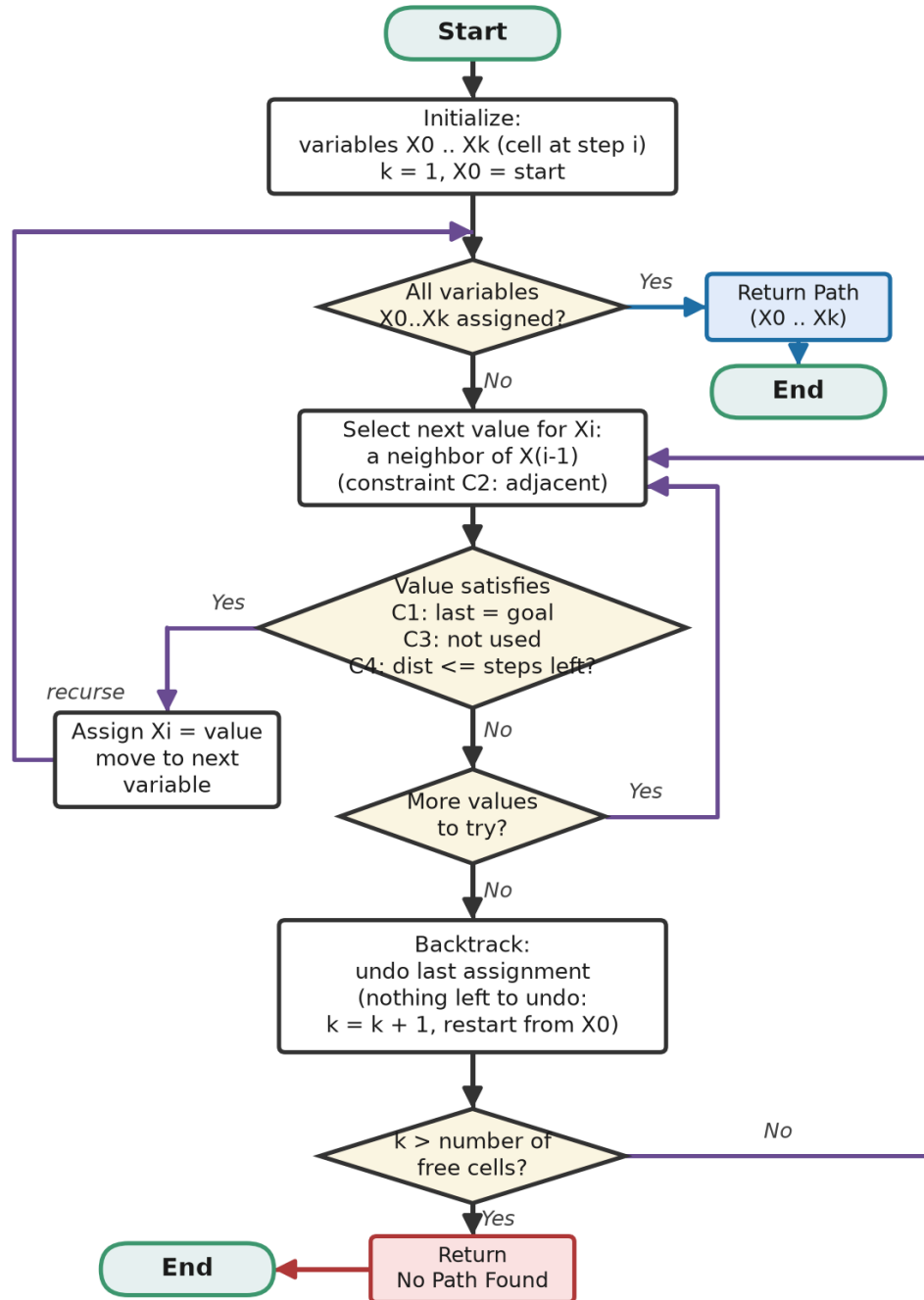


Genetic Algorithm – Optimization / CSP





CSP Backtracking Search — Constraint Satisfaction



Code

```

from collections import deque
import heapq
import random
import time

# =====
# ROBOT PATH FINDING - COMPARISON OF AI SEARCH ALGORITHMS
# =====

# Grid legend:
# 0 -> free cell
# 1 -> obstacle (blocked cell)

grid = [
    [0, 0, 0, 1],
    [0, 1, 0, 0],
    [0, 0, 0, 1],
    [0, 1, 0, 0]
]

start = (0, 0)    # initial state of the robot
goal = (3, 3)     # goal state of the robot

ROWS = len(grid)
COLS = len(grid[0])

# =====
# STATE SPACE (all free cells the robot can occupy)
# =====

state_space = [
    (r, c)
    for r in range(ROWS)
    for c in range(COLS)
    if grid[r][c] == 0
]

print("STATE SPACE:")
print(state_space)

# =====
# ACTIONS (Up, Down, Left, Right)

```

```

# =====

moves = [
    (-1, 0),    # Up
    (1, 0),     # Down
    (0, -1),    # Left
    (0, 1)      # Right
]

# =====
# SUCCESSOR FUNCTION
# =====

def get_neighbors(state):
    neighbors = []
    for dr, dc in moves:
        nxt = (state[0] + dr, state[1] + dc)
        if nxt in state_space:
            neighbors.append(nxt)
    return neighbors

# =====
# BUILD PATH FROM PARENT MAP
# =====

def build_path(parent):
    path = []
    node = goal
    while node != start:
        path.append(node)
        node = parent[node]
    path.append(start)
    path.reverse()
    return path

# =====
# HEURISTIC - MANHATTAN DISTANCE TO GOAL
# =====

def manhattan(state):
    return abs(state[0] - goal[0]) + abs(state[1] - goal[1])

```

```

# =====
# 1. BREADTH-FIRST SEARCH (Uninformed)
# =====

def bfs():
    frontier = deque([start])
    visited = {start}
    parent = {}
    explored = 0

    while frontier:
        node = frontier.popleft()
        explored += 1

        if node == goal:
            return build_path(parent), explored

        for nxt in get_neighbors(node):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = node
                frontier.append(nxt)

    return None, explored

# =====
# 2. DEPTH-FIRST SEARCH (Uninformed)
# =====

def dfs():
    frontier = [start]
    visited = {start}
    parent = {}
    explored = 0

    while frontier:
        node = frontier.pop()
        explored += 1

        if node == goal:
            return build_path(parent), explored

        for nxt in get_neighbors(node):
            if nxt not in visited:
                visited.add(nxt)

```

```

        parent[nxt] = node
        frontier.append(nxt)

    return None, explored

# =====
# 3. DEPTH-LIMITED SEARCH (Uninformed)
# =====

DEPTH_LIMIT = 6

def dls(limit=DEPTH_LIMIT):
    frontier = [(start, 0, [start])]      # (state, depth, path so far)
    explored = 0

    while frontier:
        node, depth, path = frontier.pop()
        explored += 1

        if node == goal:
            return path, explored

        if depth < limit:                  # cutoff: do not expand at depth L
            for nxt in get_neighbors(node):
                if nxt not in path:        # avoid cycles on the current path
                    frontier.append((nxt, depth + 1, path + [nxt]))

    return None, explored

# =====
# 4. ITERATIVE DEEPENING DFS (Uninformed)
# =====

def iddfs(max_depth=len(state_space)):
    total_explored = 0

    for limit in range(max_depth + 1):    # limit = 0, 1, 2, ...
        path, explored = dls(limit)
        total_explored += explored
        if path is not None:
            return path, total_explored

    return None, total_explored

```

```

# =====
# 5. A* SEARCH (Informed)   $f(n) = g(n) + h(n)$ 
# =====

def a_star():
    frontier = [(manhattan(start), 0, start)]  # (f, g, state)
    visited = set()
    parent = {}
    explored = 0

    while frontier:
        f, g, node = heapq.heappop(frontier)

        if node in visited:
            continue
        visited.add(node)
        explored += 1

        if node == goal:
            return build_path(parent), explored

        for nxt in get_neighbors(node):
            if nxt not in visited:
                new_g = g + 1
                parent[nxt] = node
                heapq.heappush(frontier, (new_g + manhattan(nxt), new_g,
nxt))

    return None, explored

# =====
# 6. HILL CLIMBING (Local Search)
# =====

def hill_climbing():
    node = start
    path = [node]
    visited = {node}
    explored = 1

    while node != goal:
        candidates = [s for s in get_neighbors(node) if s not in visited]

```

```

        if not candidates:                                # dead end
            return None, explored

        best = min(candidates, key=manhattan)

        # No neighbour is better -> stuck at a local optimum
        if manhattan(best) >= manhattan(node):
            return None, explored

        node = best
        visited.add(node)
        path.append(node)
        explored += 1

    return path, explored

```

```

# =====
# 7. GENETIC ALGORITHM (Local Search / Optimization)
# =====
# A chromosome is a list of genes, each gene is a move:
# 0 = Up, 1 = Down, 2 = Left, 3 = Right

def decode(chromosome):
    """Convert a chromosome into a path and check the constraints."""
    node = start
    path = [node]

    for gene in chromosome:
        dr, dc = moves[gene]
        r, c = node[0] + dr, node[1] + dc

        # Constraint 1: robot must stay inside the grid
        if not (0 <= r < ROWS and 0 <= c < COLS):
            return path, False

        # Constraint 2: robot cannot enter an obstacle
        if grid[r][c] == 1:
            return path, False

        node = (r, c)
        path.append(node)

    return path, True

```

```

def fitness(chromosome):
    path, valid = decode(chromosome)
    last = path[-1]
    dist = manhattan(last)

    if last == goal:                                # goal reached: shorter is better
        return 1000 + (100 - len(path))
    if valid:                                         # valid but not at goal
        return 100 - dist * 10 - len(path)
    return 10 - dist * 20                            # constraint violated: penalty

def random_chromosome(length):
    return [random.randint(0, 3) for _ in range(length)]

def initial_population(size, length):
    return [random_chromosome(length) for _ in range(size)]

def select(population):
    population.sort(key=fitness, reverse=True)
    return population[:10]                          # keep the 10 fittest

def crossover(p1, p2):
    cut = random.randint(1, len(p1) - 1)
    return p1[:cut] + p2[cut:], p2[:cut] + p1[cut:]

def mutate(chromosome, rate=0.1):
    for i in range(len(chromosome)):
        if random.random() < rate:
            chromosome[i] = random.randint(0, 3)
    return chromosome

def genetic_algorithm():
    random.seed(42)                                  # reproducible results

    POP_SIZE = 100
    CHROM_LEN = 12
    GENERATIONS = 200

    population = initial_population(POP_SIZE, CHROM_LEN)
    explored = 0

```

```

best, best_fit = None, float("-inf")

for _ in range(GENERATIONS):
    for chrom in population:
        fit = fitness(chrom)
        explored += 1
        if fit > best_fit:
            best_fit, best = fit, chrom[:]

    path, valid = decode(best)
    if path[-1] == goal and valid:
        return path, explored

    elite = select(population)
    next_gen = [elite[0][:]]          # elitism: carry the best forward

    while len(next_gen) < POP_SIZE:
        c1, c2 = crossover(random.choice(elite), random.choice(elite))
        next_gen.append(mutate(c1))
        if len(next_gen) < POP_SIZE:
            next_gen.append(mutate(c2))

    population = next_gen

return None, explored

```

```

# =====
# 8. CONSTRAINT SATISFACTION - BACKTRACKING SEARCH
# =====
# Variables : X0, X1, ..., Xk (cell occupied at step i)
# Domain    : every free cell in the state space
# Constraints:
#   C1: X0 = start and Xk = goal
#   C2: X(i+1) must be adjacent to X(i)      (one legal move)
#   C3: all Xi are different                  (no cell visited twice)
#   C4: goal must still be reachable in the remaining steps
#       (Manhattan distance <= steps left)  -> forward checking
# k is increased from 1 upwards, so the first solution is the shortest.

```

```

def csp_backtracking():
    explored = 0

    def consistent(value, assignment, step, k):
        if value in assignment:          # C3
            return False

```

```

        if manhattan(value) > k - step:                # C4
            return False
        if step == k and value != goal:                # C1 (last variable)
            return False
        return True

def backtrack(assignment, k):
    nonlocal explored
    step = len(assignment)
    if step == k + 1:
        return assignment

    for value in get_neighbors(assignment[-1]):        # C2
        explored += 1
        if consistent(value, assignment, step, k):
            result = backtrack(assignment + [value], k)
            if result is not None:
                return result
    return None                                        # backtrack

for k in range(1, len(state_space)):
    solution = backtrack([start], k)                  # C1: X0 = start
    if solution is not None:
        return solution, explored

return None, explored

# =====
# PERFORMANCE EVALUATION
# =====

def evaluate(name, algorithm):
    t0 = time.perf_counter()
    path, explored = algorithm()
    elapsed = (time.perf_counter() - t0) * 1000

    print("\n" + "=" * 70)
    print(name)
    print("=" * 70)

    if path is not None:
        length = len(path) - 1
        print("Path          :", path)
        print("Path Length    :", length)
    else:

```

```

        length = "No Path"
        print("No path found")

    print("Nodes Explored :", explored)
    print("Execution Time :", round(elapsed, 6), "ms")

    return [name, length, explored, round(elapsed, 6)]

algorithms = [
    ("BFS - Uninformed Search", bfs),
    ("DFS - Uninformed Search", dfs),
    ("DLS - Uninformed Search", dls),
    ("IDDFS - Uninformed Search", iddfs),
    ("A* - Informed Search", a_star),
    ("Hill Climbing - Local Search", hill_climbing),
    ("Genetic Algorithm - Local Search", genetic_algorithm),
    ("Backtracking - Constraint Satisfaction", csp_backtracking),
]

results = [evaluate(name, fn) for name, fn in algorithms]

# =====
# FINAL PERFORMANCE COMPARISON
# =====

print("\n\n" + "=" * 90)
print("FINAL PERFORMANCE COMPARISON")
print("=" * 90)
print(f"{'Algorithm':<40}{'Path Length':<15}{'Nodes Explored':<20}{'Time (ms)':<15}")
print("-" * 90)
for name, length, explored, t in results:
    print(f"{'name':<40}{'str(length):<15'}{'explored:<20'}{'t:<15}'")
print("=" * 90)

```

Output

```

STATE SPACE:
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 2), (1, 3), (2, 0), (2, 1), (2, 2), (3,
0), (3, 2), (3, 3)]

```

```

=====
BFS - Uninformed Search

```

```
=====
Path          : [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3)]
Path Length   : 6
Nodes Explored : 12
Execution Time : 0.016417 ms
```

```
=====
DFS - Uninformed Search
=====
Path          : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)]
Path Length   : 6
Nodes Explored : 11
Execution Time : 0.010334 ms
```

```
=====
DLS - Uninformed Search
=====
Path          : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)]
Path Length   : 6
Nodes Explored : 10
Execution Time : 0.009209 ms
```

```
=====
IDDFS - Uninformed Search
=====
Path          : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)]
Path Length   : 6
Nodes Explored : 53
Execution Time : 0.029875 ms
```

```
=====
A* - Informed Search
=====
Path          : [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3)]
Path Length   : 6
Nodes Explored : 12
Execution Time : 0.014875 ms
```

```
=====
Hill Climbing - Local Search
=====
No path found
Nodes Explored : 4
Execution Time : 0.007 ms
=====
```

Genetic Algorithm - Local Search

=====

No path found

Nodes Explored : 20000

Execution Time : 33.44375 ms

=====

Backtracking - Constraint Satisfaction

=====

Path : [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3)]

Path Length : 6

Nodes Explored : 23

Execution Time : 0.014625 ms

Final Performance Comparison

Algorithm	Category	Path Length	Nodes Explored	Time (ms)
BFS	Uninformed	6	12	0.016417
DFS	Uninformed	6	11	0.010334
DLS (L = 6)	Uninformed	6	10	0.009209
IDDFS	Uninformed	6	53	0.029875
A*	Informed	6	12	0.014875
Hill Climbing	Local Search	No Path	4	0.007
Genetic Algorithm	Local Search	No Path	20000	33.44375
Backtracking	Constraint Satisfaction	6	23	0.014625

Result

Six of the eight algorithms (BFS, DFS, DLS, IDDFS, A* and CSP Backtracking) found a valid 6-step path from (0, 0) to (3, 3), which is the shortest possible. DLS with limit $L = 6$ explored the fewest nodes (10) because the limit matched the solution depth exactly. IDDFS explored the most among the successful methods (53) because it repeats the shallow levels on every iteration, but it needs no preset limit. BFS and A* explored all 12 free cells; CSP Backtracking checked 23 values while it increased the path length k from 1 to 6.

Hill Climbing failed after 4 nodes. It moved greedily $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (3, 0)$, reaching a corner whose only other neighbour, $(3, 1)$, is an obstacle. With no unvisited neighbour and no backtracking, it got stuck even though the goal was reachable. The Genetic Algorithm evaluated 20,000 chromosomes (100 per generation for 200 generations) in about 33.4 ms, but no chromosome reached the goal without breaking a constraint.

Conclusion

On this 4×4 grid, the systematic methods were the most reliable: the uninformed searches (BFS, DFS, DLS, IDDFS), the informed A* search and CSP Backtracking all reached the goal with a 6-step path. DLS was the cheapest when given the right limit, and IDDFS found the same path without knowing the limit in advance. Hill Climbing got trapped in a dead end because it never backtracks, and the Genetic Algorithm could not produce a valid path with the chosen population size, chromosome length and fitness function.

Outcomes

- Implemented eight search algorithms for robot path finding and compared them on path length, nodes explored and execution time.
- BFS, DFS, DLS, IDDFS, A* and CSP Backtracking each found a 6-step path in under 0.03 ms; DLS explored the fewest nodes (10).
- Modelling the path as a CSP showed how constraints and forward checking prune the search while still guaranteeing the shortest path.
- Hill Climbing showed the main weakness of local search: greedy moves with no backtracking can lead into a dead end. The Genetic Algorithm showed that optimization methods need parameter tuning and cost far more computation.
- For a small, fully known grid, classical, heuristic and constraint-based search are the better choice.

GitHub link: https://github.com/Harshini-Bhandary21/AIML_LAB_PRACTICALS